

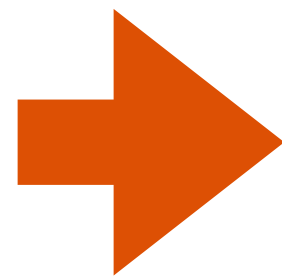
Web Clients

*“Simplicity is the
ultimate sophistication.”*

Leonardo Da Vinci

Zusatzaufgaben

Drehbuch (initial)



1 Drehbuch, Intro, Classic MVC	
2 Standortbestimmung (keine Vorlesung)	
3 Attributes	
4 Master-Detail	UI Architecture
5 Catching up	
6 Projector Pattern	
7 CSS Power Fueling (?)	Custom UI Elements
8 SVG and Canvas	
9 Animation	
10 Big_UI_FW(TM) Integration	
11 Connecting to REST	
12 Service Abstraction	
13 Reactive, Stable Binding	Remoting
14 Applied Concepts	
15 Team App: Putting it all together	

From Zero to Hero 1

Web Programming:
Observable, ObservableList, Scheduler
Testing
Classic MVC

From Zero to Hero 2

Attribute

Converter, Validator

Reichhaltige Attribute

Spezialisierte Projektoren

Master-Detail

Klassisch: Liste plus Formular

Unoffensichtliche Master-Detail Views

Wie mit der Selektion umgehen?
(wir machen es erstmal falsch...)

Checkliste

Alle Tests grün

Alle nötigen Tests vorhanden

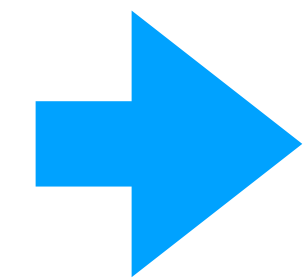
Demo läuft

Architekturkonzepte eingehalten

Performance, Speicherleaks?

Concurrency, Hysterese?

From Zero to Hero 3



Allgemeines Projector Pattern

[https://dierk.github.io/Home/
projectorPattern/ProjectorPattern.html](https://dierk.github.io/Home/projectorPattern/ProjectorPattern.html)

Projector Pattern

efficient creation of ambitious user interfaces

Dierk König

Context

User interfaces with potentially many screens that can be classified into typical schemes

Widget creation and binding through UI toolkit

Presentation model abstraction

Problem

- Implementing dedicated views for many screens including model binding leads to much code that needs to be created, tested, and maintained.
- Moreover, this code is UI-toolkit specific and needs to be replaced with any change of the toolkit. The sheer amount of code can make this prohibitively expensive.

Forces

Static code generation and templating approaches often don't yield enough flexibility in the view construction:

- the model binding & interaction paradigm

- dynamic modifications and extensions

- compositional aggregates

Even more importantly, statically created structures are generally difficult to re-apply when the structure needs modification



presentation

projection

presentation
models



projector

Solution



Rich Presentation Models / Attributes

- Abstract Factory (GOF): **IProjector** Interface
 - specify **Projection Methods**
- Caller programs against the interface only

Richness

- Attributes: Value, BaseValue, -> dirty
- Bindable Tags: Label, Visible, Optional, Enabled, Editable, Regex, Valid, ValidationError, HelpUrl, Tooltip, WidgetHint, <custom>

Solution

- Rich Presentation Models / Attributes



Abstract Factory (GOF): **IProjector** Interface

- specify **Projection Methods**
- Caller programs against the interface only

Projector Types



```
/**
 * Interface for {@link InputProjectionType}s.
 * @typedef IInputProjector
 * @property { ChangeInputProjectionType } projectChangeInput
 * @property { InstantInputProjectionType } projectInstantInput
 * @property { DebounceInputProjectionType } projectDebounceInput
 */

/**
 * @typedef { <_T_>
 *   (inputController: !SimpleInputControllerType<_T_>,
 *     formCssClassName: !String )
 *   => [HTMLLabelElement, HTMLInputElement]
 * } InputProjectionType
 */
```

Projector Implementation



```
/**  
 * Namespace object for the {@link IInputProjector} functions.  
 * @type { IInputProjector }  
 */  
const InputProjector = {  
  projectInstantInput,  
  projectChangeInput,  
  projectDebounceInput  
};
```

Projector Use



```
import { InputProjector } from "./simpleInputProjector.js";  
  
// ...  
  
const [labelElement, spanElement] =  
    InputProjector.projectInstantInput(inputController, "ListItem");
```

Composite Structure

The screenshot displays the SAP Composite Structure configuration interface. The main window is titled "C2E - Employee Portfolio Manager (DRAFT)" and "A0Z - Supervisor (ACTIVE)". It shows a hierarchical structure of components and rule sets.

IM Portfolio Management Cockpit (C2E) - S2P - Employee Portfolio Manager

State	DRAFT	Version	1.0.0	Classification	CUSTOMIZABLE
Target Mandator	S2P - UBS IM Switzerland	Target Environment	ALL	Owner Mandator	S1P

Cockpit Details:

Cockpit	C2E - IM Portfolio Management Cockpit	Release	1
Target Mandator	S2P - UBS IM Switzerland	Target Environment	ALL
Profile	C2E-Profile	Entitlement (access right)	Employee Portfolio Manager

Rule Sets

Rule Set 1 is selected. It contains two options:

- ☒ Personnel : Grants access to all categories of clients including personnel
- ☐ Non-Personnel : Grants access to all categories of clients excluding personnel

Functions and Views

Client Category

Active Object

Authorization Panel (Right):

Allowed functions and views for the authorization:
C2E, C2E-PROFIL, EMPLOYEE PORTFOLIO MANAGER

Buttons: [Filter] [Excel] [Save] [Default] [Refresh]

Table of functions and views:

Name
ActiveRepositoryMgmtAdmin
ClientInstructionAccess
ClientInstructionRead
ClientRead

Rows: 4

Behavior: Validation

C2E - Employee Portfolio Manager (DRAFT) A0Z - Supervisor (ACTIVE) *OpenDolphin is Groovy (DRAFT) *F2F -- (DRAFT) X

IM Reporting Cockpit (F2F) - S1P - -

State	DRAFT	Version	1.0.0	Classification	CUSTOMIZABLE	
Target Mandator	S1P - UBS Mandate Business Headquarter		Target Environment	ALL	Owner Mandator	S1P

Cockpit	F2F - IM Reporting Cockpit	Release	1
Target Mandator	S1P - UBS Mandate Business Headquarter	Target Environment	ALL
Profile		Entitlement (access right)	

▼ Rule Sets 1 2 3 4 + -

+ -

Region	☒ Allow : Access to data items allocated to any OU ☐ MyOwnOU : Access to data items allocated to the deployment OU of the user ☐ MyOwnOU+ : Access to data items allocated to the deployment OU of user and below
--------	--

► Active Object

Flexibility

C2E - Employee Portfolio Manager (DRAFT) A0Z - Supervisor (ACTIVE) *OpenDolphin is Groovy (DRAFT) x

OpenDolphin is Groovy

Model Type Client Category Rule Type MATCH Owner Mandator S1P

Identity Name RuleDef.checkJavaLand Name OpenDolphin is Groovy

Target Mandator S1P - UBS Mandate Business Headquarter Target Environment ALL

Model Type Client Category Condition ILIKE

Rule Type MATCH

► Values (17)

► Description: 'CU @ Canoo booth'

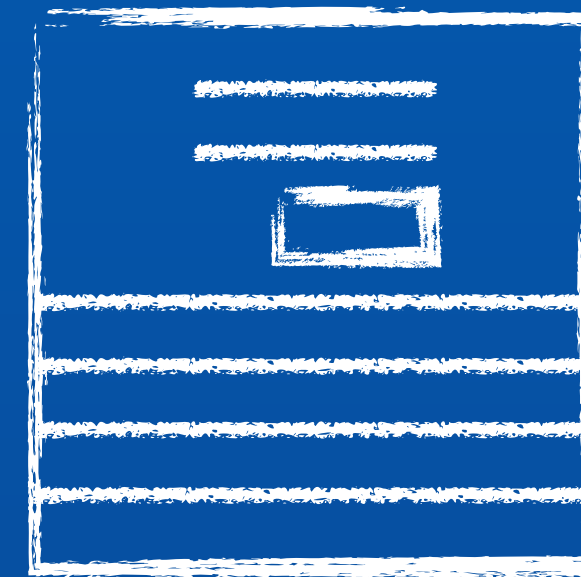
User Explorer User Group Explorer Authorization Explorer Rule Definition Explorer Functions/Views Explorer Reference Data Explorer

Code Domain	Code Value	Description	Short Description	Language	Long Description
R11003.3505	13	-	-	en	Personnel: wealth managem...
R11003.3505	14	-	-	en	Personnel: Wealth Managem...
R11003.3505	15	-	-	en	Self Directed Private Clients (...)
R11003.3505	16	-	-	en	Special Clients (NALO and U...

Rows: 17

Your Catalog

Simple Form, 2-Column Form, Search Form,
Navigation, Master-Detail, Foldable Groups,...



Not only layout but also **Interaction Paradigm!**

Resulting Context

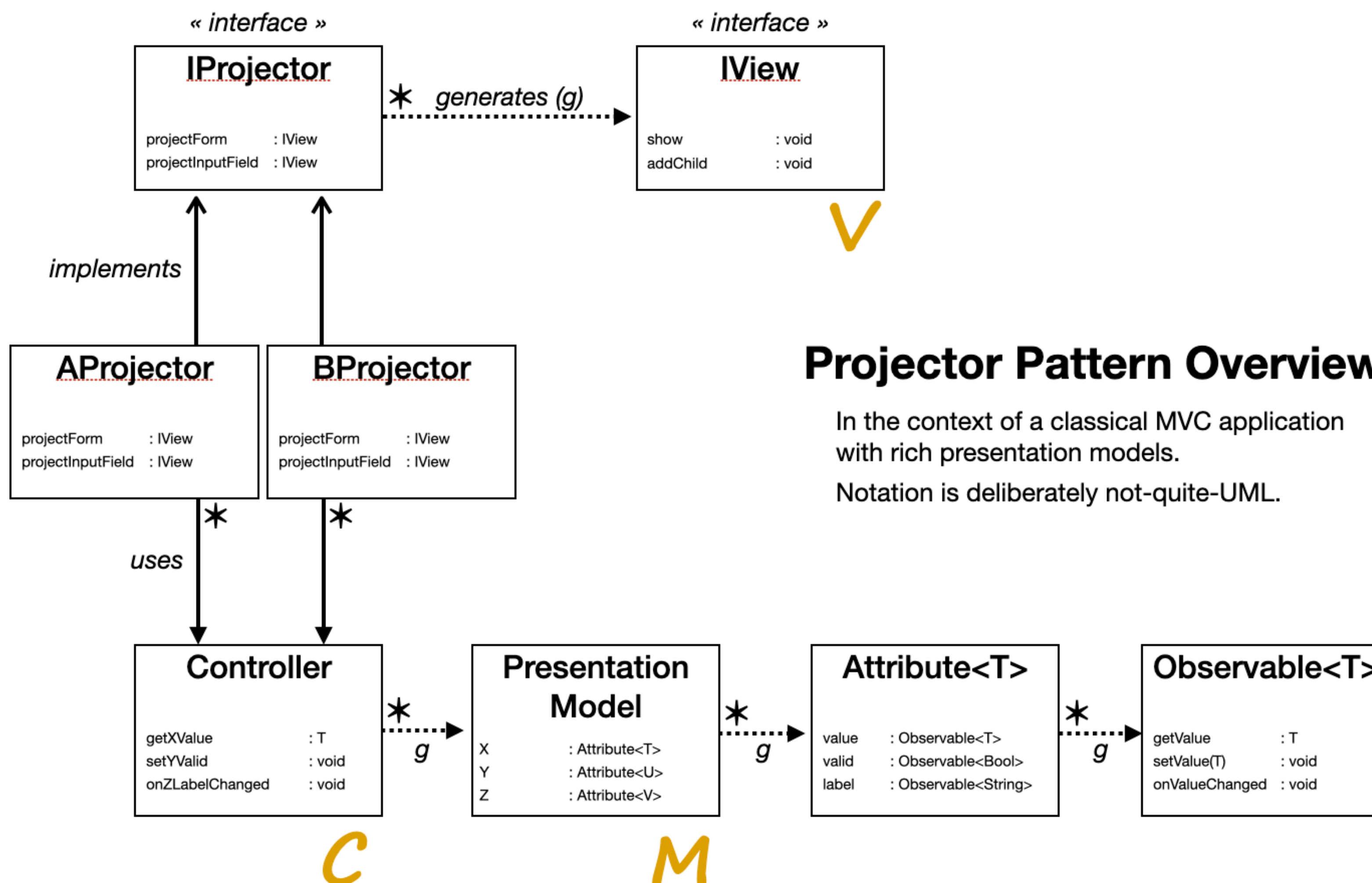
- Simple view creation by selecting a projection from a catalog
- Exchangeable Projectors enable quick and safe migration as well as multi-channel support
- Dramatically reduced code size, less structural duplication, reduced testing effort, and fewer bugs
- General, parameterized Projectors can be more complex to implement and test

Known Uses

- Kolibri
- OpenDolphin / Dolphin Platform
- UBS STMP
- KBS KKVS 4
- Various P5/P6 Projects (GroupHub, DuploFX, Big Lazy Table, Navigation, Logging Control, Facetted Search)

Related Patterns

- Components
Dependency to data is reversed.
Can be used as projection target.
- Abstract UI Toolkit
Applicable in a limited context, where
least common denominator between all
UI technologies is OK



Projector Pattern Overview

In the context of a classical MVC application with rich presentation models.

Notation is deliberately not-quite-UML.

<https://dierk.github.io/Home/projectorPattern/ProjectorPattern.html>

Aufgaben

- 1) Kurzpräsentation `display:flex`
- 2) Kurzpräsentation `display:grid`

Jeweils Hauptanwendungsfall mit
Codebeispiel
(keine exotischen Varianten)

Aufgaben (in 2 Wochen)

- 1) Baue einen Projektor, der die Liste der Personen als HTML table anzeigt mit Visualisierung der selektierten table row.
- 2) Das gleiche mit `<div class=row>` und subgrid.